



UART HAL Interface User Guide

Issue V1.1
Date 25-Jan-2024

Copyright © UNISOC (Shanghai) Technologies Co., Ltd. All rights reserved.

All data and information contained in or disclosed by this document is confidential and proprietary information of UNISOC (Shanghai) Technologies Co., Ltd. (hereafter referred as UNISOC) and all rights therein are expressly reserved. This document is provided for reference purpose, no license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document, and no express and implied warranties, including but without limitation, the implied warranties of fitness for any particular purpose, and non-infringement, as well as any performance. By accepting this material, the recipient agrees that the material and the information contained therein is to be held in confidence and in trust and will not be used, copied, reproduced in whole or in part, nor its contents revealed in any manner to others without the express written permission of UNISOC. UNISOC may make any changes at any time without prior notice. Although every reasonable effort is made to present current and accurate information, UNISOC makes no guarantees of any kind with respect to the matters addressed in this document. In no event shall UNISOC be responsible or liable, directly or indirectly, for any damage or loss caused or alleged to be caused by or in connection with the use of or reliance on any such content.

Please refer to the UNISOC Documents in the UNISOC Deliverables for the use of the Deliverables. Any loss caused by the modification, customization or use of the UNISOC Deliverables in violation of the instructions in the UNISOC Documents shall be undertaken by those who conduct so. The performance indicators, test results and parameters in the UNISOC Deliverables are all obtained in the internal development and test system of UNISOC and are only for the reference. Before using UNISOC Deliverables commercially or conducting mass production of the Deliverables, comprehensive testing and debugging in combination with its own software and hardware test environment are pre-requisite.

Trademarks

UNISOC, , Spreadtrum, SPRD, RDA, and other UNISOC trademarks are trademarks of UNISOC (Shanghai) Technologies Co., Ltd. and/or its affiliated companies.

All other trademarks and trade names mentioned in this document are the property of their respective owners.

Disclaimer

This document may include third-party content, including but not limited to third-party information, software, components, and data. UNISOC does not control or assume responsibility for any third-party content, including but not limited to its accuracy, compatibility, reliability, availability, legitimacy, appropriateness, performance, non-infringement, and status update, unless otherwise specified in this document. UNISOC does not provide any guarantee or authorization for the third-party content mentioned or referenced in this document.

User should check the availability of third-party content. If a third-party license is needed, user shall obtain it in an authorized or legal way, unless otherwise specified in this document.

UNISOC (Shanghai) Technologies Co., Ltd.



About this document

Purpose

This document introduces the commonly-used interfaces and related data structures of UART in the HAL, including data structures, interface functions, driver calling process and FAQ.

This document is applicable in chip platforms including UWS6130E, UWS6131E, UWS6133, UWS6133E and UWS6135E.

Intended audience

This document is intended primarily for software developers and testers who need to use the UART interfaces and data structures provided in the HAL.

Acronyms and abbreviations

Acronym and abbreviation	Full name
APB	Advanced Peripheral Bus
CTS	Clear to Send
DMA	Dynamic Multithreading Architecture
FIFO	First In First Out
HAL	Hardware Abstract Layer
RTS	Request To Send
UART	Universal Asynchronous Receiver/Transmitter

Symbol conventions

The symbols that may be found in this guide are defined in the following table.

Symbol	Description
 NOTE	Calls attention to important information, best practices and tips. NOTE is used to address information not related to personal injury, equipment damage, and environment deterioration.
 CAUTION	Calls attention to error-prone operations. CAUTION is used to address information not related to personal injury, equipment damage,

Symbol	Description
	and environment deterioration.
 WARNING	Calls attention to irreversible operations. WARNING is used to address information not related to personal injury and environment deterioration.

Revision history

Issue	Date	Description
V1.1	25-Jan-2024	- Add 4 Driver calling process and 5 FAQ. - Update API. - Optimize content description.
V1.0	03-Jan-2020	This issue is the first official release

Keywords

UART, HAL, API

Contents

1 Overview	1
2 Data structure	2
2.1 Macro	2
2.2 Enum	2
2.2.1 uart_event	2
2.3 Struct	3
2.3.1 uart	3
2.3.2 uart_port	3
2.3.3 sprd_uart	4
2.3.4 uart_ops	6
2.3.5 termios	7
2.3.6 UART_INIT_PARA_T	9
3 API	10
3.1 uart_port_open	10
3.2 uart_port_param_init	10
3.3 uart_set_termios	11
3.4 uart_write	11
3.5 uart_read	12
3.6 uart_port_close	12
3.7 uart_get_ttfifo_len	13
3.8 uart_get_rtfifo_len	13
3.9 uart_start_tx	14
3.10 uart_stop_tx	14
3.11 uart_start_rx	15
3.12 uart_stop_rx	15
3.13 uart_dump_reg	16
4 Driver calling process	17
4.1 Calling steps	17
4.2 Calling example	17
5 FAQ	19
5.1 Many UART interrupts cause Kernel Crash	19
5.2 UART outputs garbled code	19
5.3 UART fails to send data normally	19

List of figures

Figure 1-1 The position of UART HAL in the system framework	1
--	---

List of tables

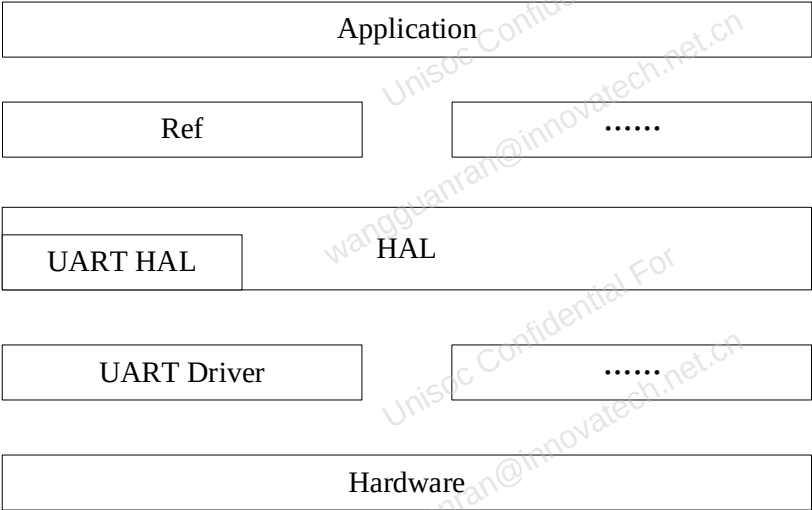
Table 2-1 Definition of bit[7:0] 8

Table 5-1 The performance parameters of UART flow control 20

1 Overview

The position of UART Hardware Abstract Layer (HAL) in the system framework is shown as [Figure 1-1](#).

Figure 1-1 The position of UART HAL in the system framework



The HAL provides all external interfaces of UART. The upper layer calls the UART-related interfaces in the HAL to configure the UART controller, to control UART to complete related operations.

2

Data structure

This chapter mainly introduces UART-related data structures, including macros, enums and structs.

2.1 Macro

Macro name	Description
UART_DEBUG	Call <code>uart_dump_reg</code> to get the UART register information.

2.2 Enum

2.2.1 `uart_event`

Function

Define two event types for UART reading and writing to inform the user of the current UART state by passing relevant events.

Definition

```
enum uart_event {  
    EVENT_READ,  
    EVENT_WRITE,  
};
```

Member Description

Member	Description
<code>EVENT_READ</code>	Indicate that UART performs a reading operation.
<code>EVENT_WRITE</code>	Indicate that UART performs a writing operation.

2.3 Struct

2.3.1 uart

Function

Define the **uart_port** struct and the maximum number of UARTs supported.

Definition

```
struct uart {  
    struct uart_port *up;  
    int count;  
};
```

Member Description

Member	Description
up	The struct pointer of uart_port.
count	The maximum number of UARTs supported.

2.3.2 uart_port

Function

Define the struct (**sprd_uart**) that configures UART parameters, UART operation functions and other related content.

Definition

```
struct uart_port {  
    struct sprd_uart *su;  
    struct uart_ops *ops;  
    struct termios termios;  
    BOOLEAN suspend;  
    BOOLEAN init;  
};
```

Member Description

Member	Description
su	The struct pointer of sprd_uart.
ops	The function pointer of UART operation.

Member	Description
termios	UART parameter configuration.
suspend	UART suspension state. - TRUE: UART is suspended. - FALSE: UART is not suspended.
init	UART initialization state. - TRUE: UART is initialized. - FALSE: UART is not initialized.

2.3.3 sprd_uart

Function

Define UART parameters.

Definition

```

struct sprd_uart {
    int index;
    uint32 base;
    int irq;

    /*apb eb*/
    uint32 apb_base;
    uint32 eb;
    uint32 reg_offset;

    /* source clock config */
    uint32 sclk;
    uint32 sclk_base;
    uint32 sclk_bit;

    /* dma mode */
    uint32 dma_mode;
    uint32 tx_id;
    uint32 rx_id;

    uint32 baud;

```

```

UART_CB cb;
uint32 tx_chn;
uint32 rx_chn;

int irq_cnt;
int write_cnt;
};

```

Member Description

Member	Description
index	UART ID.
base	The base address of UART register.
irq	UART interrupt ID.
apb_base	The Advanced Peripheral Bus (APB) base address of enabling UART.
eb	The APB bit of enabling UART.
reg_offset	The offset address of UART.
sclk	UART clock source.
sclk_base	The selected address of UART clock source.
sclk_bit	The selected bit of UART clock source.
dma_mode	Dynamic Multithreading Architecture (DMA) transmission mode. 0 indicates the Non-DMA mode 1 indicates the DMA mode
tx_id	UART TX DMA request ID.
rx_id	UART RX DMA request ID.
baud	UART baud rate, unit: bit/s.
cb	UART callback function.
tx_chn	UART TX DMA channel ID.
rx_chn	UART RX DMA channel ID.
irq_cnt	UART interrupt count.
write_cnt	UART writing count.

2.3.4 uart_ops

Function

Define UART operation functions.

Definition

```
struct uart_ops {
    int (*lower_init)(struct sprd_uart *, struct termios *);
    int (*init)(struct sprd_uart *, UART_CB);
    int (*shutdown)(struct sprd_uart *);
    int (*set_termios)(struct sprd_uart *, struct termios *);
    int (*get_termios)(struct sprd_uart *, struct termios *);
    uint32 (*tx)(struct sprd_uart *, uint8 *, uint32);
    uint32 (*rx)(struct sprd_uart *, uint8 *, uint32);
    int (*get_tfifo_len)(struct sprd_uart *);
    int (*get_rfifo_len)(struct sprd_uart *);
    int (*start_tx)(struct sprd_uart *);
    int (*stop_tx)(struct sprd_uart *);
    int (*start_rx)(struct sprd_uart *);
    int (*stop_rx)(struct sprd_uart *);
    int (*apb_enable)(struct sprd_uart *);
};
```

Member Description

Member	Description
lower_init	The function pointer of initializing basic UART functions.
init	The function pointer of initializing full UART functions.
shutdown	The function pointer of closing UART.
set_termios	The function pointer of setting UART parameters.
get_termios	The function pointer of getting UART parameters.
tx	The function pointer of UART writing.
rx	The function pointer of UART reading.
get_tfifo_len	The function pointer of getting the number of data stored in TX First In First Out (FIFO) for UART.
get_rfifo_len	The function pointer of getting the number of data stored in RX FIFO for UART.
start_tx	The function pointer of enabling TX interrupt for UART.

Member	Description
stop_tx	The function pointer of disabling TX interrupt for UART.
start_rx	The function pointer of enabling RX interrupt for UART.
stop_rx	The function pointer of disabling RX interrupt for UART.
apb_enable	The pointer of UART APB enable switch.

2.3.5 termios

Function

Define the baud rate and parameters such as data bits, stop bits, parity bits, hardware flow control and DMA mode for initializing UART.

Definition

```
struct termios {
    /* controller flag, the low 8 bit for baudrate */
    uint32 baud;
    /* enable dma mode, hw flow ctrl */
    uint32 cflag;
};
```

Member Description

Member	Description
baud	Configure UART baud rate, unit: bit/s.
cflag	Configure UART parameters such as data bits, stop bits, parity bits, hardware flow control and DMA mode. cflag has 32 bits. Currently, only the lower 8 bits are used and the others are reserved. The definition of bit[7:0] is shown in Table 2-1. The macro definitions related to bit[7:0] in cflag are as follows: <pre>#define C_DMA 0x00000001 //Enable the DMA mode for UART #define CRX_RTS 0x00000002 //Enable hardware flow control in data receiving #define CTX_CTS 0x00000004 //Enable hardware flow control in data sending #define CSIZE 0x00000018 //Control data bit width, and the data bit width is 8 bits by default #define CS5 0x00000000 //5 data bits #define CS6 0x00000000 //6 data bits #define CS7 0x00000000 //7 data bits #define CS8 0x00000000 //8 data bits</pre>

Member	Description
	<pre>#define CSSTOPB 0x00000020 //2 stop bits #define PAREN_DIS 0x00000000 //No parity bit #define PARENB 0x00000040 //parity enabling bit #define PAREVEN 0X00000000 //even parity, default value #define PARODD 0x00000080 //odd parity</pre>

Table 2-1 Definition of bit[7:0]

BIT	Description
BIT0	Configure DMA transmission mode. 0 indicates to disable DMA transmission mode. 1 indicates to enable DMA transmission mode.
BIT1	Configure hardware flow control in data receiving. 0 indicates to disable hardware flow control in data receiving. 1 indicates to enable hardware flow control in data receiving.
BIT2	Configure hardware flow control in data sending. 0 indicates to disable hardware flow control in data sending. 1 indicates to enable hardware flow control in data sending.
BIT[4:3]	Configure data bit width. 00 indicates the data bit width is configured as 5 bits. 01 indicates the data bit width is configured as 6 bits. 10 indicates the data bit width is configured as 7 bits. 11 indicates the data bit width is configured as 8 bits.
BIT5	Configure stop bits. 0 indicates one stop bit. 1 indicates two stop bits.
BIT6	Configure the parity enabling bit. 0 indicates to disable the parity bit. 1 indicates to enable the parity bit.
BIT7	Configure the parity bit. 0 indicates even parity. 1 indicates odd parity.

Configuration example

When DMA transmission is disabled for UART, and there is no hardware flow control, 8 data bits, 1 stop bit and no parity bit, cflag is configured as follows:

cflag |= CS8 | PAREN_DIS

2.3.6 UART_INIT_PARA_T

Function

Define the parameters for dynamically modifying UART.

Definition

typedef struct

```
{
    uint16 tx_watermark;
    uint16 rx_watermark;
    uint32 baud_rate;
    UART_PARITY_E    parity;
    UART_PARITY_SET_E    parity_enable;
    UART_BYTE_SIZE_E    byte_size;
    UART_STOP_BIT_E    stop_bits;
    UART_FLOW_CTR_E    flow_control;
    UART_DS_CTRL_E    ds_wakeup_en;
    UART_DS_TYPE_E    ds_wakeup_type;
    UART_MODE_E    dma_mode;
} UART_INIT_PARA_T;
```

Member Description

Member	Description
tx_watermark	TX FIFO empty threshold.
rx_watermark	RX FIFO full threshold.
baud_rate	Baud rate, unit: bit/s.
parity	Parity bit.
parity_enable	Parity enabling bit.
byte_siz	Data bit.
stop_bits	Stop bit.
flow_control	Flow control configuration.
ds_wakeup_en	Wakeup enabled.
ds_wakeup_type	Wakeup mode selection.
dma_mode	DMA function configuration.

3 API

This chapter introduces the commonly-used external interface functions of UART in the HAL and provides a detailed description of their functions.

3.1 uart_port_open

Function

This function is used to enable the UART controller, initialize UART, register interrupts and callback functions.

Definition

```
PUBLIC int uart_port_open(int n, UART_CB cb);
```

Parameter description

Parameter	Description
n	UART ID.
cb	UART callback function. When UART interrupt detects a reading/writing event, a callback function is passed in to pass the relevant events to the upper layer, so that the upper layer can know the current UART state and take the next action.

Return value

- Successful: 0.
- Failed: a negative number.

3.2 uart_port_param_init

Function

This function is used to identify the relevant parameters of the UART controller during driver initialization, including baud rate, parity check and flow control, assign the parameters to the **termios** struct, and call **uart_set_termios** to ultimately achieve the parameter configuration.

Definition

```
PUBLIC int uart_port_param_init(int n, UART_INIT_PARAM *param);
```

Parameter description

Parameter	Description
n	UART ID.
param	The struct pointer of UART parameter configuration.

Return value

- Successful: 0.
- Failed: a negative number.

3.3 uart_set_termios

Function

This function is used to configure the relevant parameters of the UART controller, including baud rate, parity check and flow control.

Definition

```
PUBLIC int uart_set_termios(uint32 n, struct termios *termios);
```

Parameter description

Parameter	Description
n	UART ID.
termios	The struct pointer of UART parameter configuration.

Return value

- Successful: 0.
- Failed: a negative number.

3.4 uart_write

Function

This function is used to write UART data.

Definition

```
PUBLIC uint32 uart_write(uint32 n, const uint8 *buf, uint32 len);
```

Parameter description

Parameter	Description
n	UART ID.
buf	The buffer pointer of the data to be sent.
len	The length of the data to be sent, unit: Byte.

Return value

- Successful: the number of sent data.
- Failed: 0.

3.5 uart_read

Function

This function is used to read UART data.

Definition

```
PUBLIC uint32 uart_read(uint32 n,uint8 *buf, uint32 len);
```

Parameter description

Parameter	Description
n	UART ID.
buf	The buffer pointer of the data to be received.
len	The length of the data to be received, unit: Byte.

Return value

- Successful: the number of read data.
- Failed: 0.

3.6 uart_port_close

Function

This function is used to close UART.

Definition

```
PUBLIC void uart_port_close(int n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

None.

3.7 uart_get_tfifo_len

Function

This function is used to get the number of unsent data stored in the UART TX FIFO.

Definition

```
PUBLIC uint32 uart_get_tfifo_len(uint32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: the number of unsent data stored in the UART TX FIFO.
- Failed: 0.

3.8 uart_get_rfifo_len

Function

This function is used to get the number of unread data stored in the UART RX FIFO.

Definition

```
PUBLIC uint32 uart_get_rfifo_len(uint32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: the number of unread data stored in the UART RX FIFO.
- Failed: 0.

3.9 uart_start_tx

Function

This function is used to enable the UART TX interrupt.

Definition

```
PUBLIC int uart_start_tx(uint32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: 0.
- Failed: a negative number.

3.10 uart_stop_tx

Function

This function is used to disable UART TX interrupt.

Definition

```
PUBLIC uint32 uart_stop_tx (uint32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: 0.
- Failed: a negative number.

3.11 uart_start_rx

Function

This function is used to enable the UART RX interrupt.

Definition

```
PUBLIC uint32 uart_start_rx (uint32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: 0.
- Failed: a negative number.

3.12 uart_stop_rx

Function

This function is used to disable UART RX interrupt.

Definition

```
PUBLIC uint32 uart_stop_rx (uint32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: 0.
- Failed: a negative number.

3.13 uart_dump_reg

Function

This function is used to get the UART register information.

Definition

```
int uart_dump_reg(unit32 n);
```

Parameter description

Parameter	Description
n	UART ID.

Return value

- Successful: 0.
- Failed: a negative number.

Notice

This interface can be used only if the **UART_DEBUG** macro is enabled.

4 Driver calling process

This chapter mainly introduces the software process for calling UART to work normally.

4.1 Calling steps

- Step 1** Call the **uart_port_open** interface function to initialize UART and pass the callback function.
- Step 2** Call **uart_port_param_init** or **uart_set_termios** interface function to configure the parameters including UART baud rate, empty and full threshold, frame format and transmission mode.
- Step 3** Call the **uart_read/uart_write** interface function to read/write data.
- Step 4** Call the **uart_port_close** interface function to close the corresponding UART ports.

----End

4.2 Calling example

This section takes “Configuring UART parameters and writing data through UART” as an example to illustrate the UART calling process.

```
void uart_callback(unsigned int evt)
{
    /* nothing to do */
}

void uart_test(uint32 port)
{
    char buf[] = "debug uart write"
    UART_INIT_PARA_T param;
    uart_port_open(port, uart_callback);
    param.baud_rate = 115200;
    param.stop_bits = ONE_STOP_BIT;
    param.parity_enable = PARITY_DISABLE;
    param.byte_size = EIGHT_BITS;
    param.flow_control = NO_FLOW_CONTROL;
    param.dma_mode = UART_DMA_MODE_MAX;
    param.ds_wakeup_en = DS_WAKEUP_DISABLE;
    uart_port_param_init(port, &param);
```

```
uart_write(port, buf, sizeof(buf));  
uart_port_close(port);  
}
```

5 FAQ

5.1 Many UART interrupts cause Kernel Crash

Issue description

UART has many interrupts, more than 1000 times per second.

Possible cause

A large number of break interrupts is generally generated as the UART RX pin being continuously pulled down externally.

Solution

Ensure that RX is in pull-up state.

5.2 UART outputs garbled code

Issue description

The UART port outputs garbled code.

Possible cause

It is generally caused by incorrect baud rate settings.

Solution

1. Check whether the serial port baud rate configuration at both ends of the communication are consistent.
2. Check whether the pull-up and pull-down configuration of pins are correct. Generally, the pull-up function of the pins that work as UART need to be enabled.
3. Check the hardware to avoid external interference.

NOTE

For any questions, contact UNISOC FAE for assistance.

5.3 UART fails to send data normally

Issue description

UART fails to send data normally.

Possible cause

It is generally because the hardware flow control is enabled. When Clear to Send (CTS) pin is pulled high by the remote Request To Send (RTS) pin, the data is not transmitted.

For the performance parameters of UART flow control, see [Table 5-1](#).

Table 5-1 The performance parameters of UART flow control

Input clock frequency (MHz)	Maximum rate supported (bit/s)
26	• With flow control: 3250000 • Without flow control: 115200
48	• With flow control: 4000000 • Without flow control: 115200
64	• With flow control: 4000000 • Without flow control: 115200
51.2	• With flow control: 4000000 • Without flow control: 115200
96	• With flow control: 4000000 • Without flow control: 115200

Solution

1. Check whether the pin is configured normally.
2. Check whether the remote device continues to receive data. If not, the transmission stops.

NOTE

For any questions, contact UNISOC FAE for assistance.